



Università degli Studi di Palermo

System Design Document
Ingegneria Informatica – AA 2025/2026

A cura di:
Gabriele Lapi
Matteo Lopez
Giorgio Seminara
Dennis Secco

Sommario

1	<i>Introduzione</i>	3
1.1	Compromessi e Scelte nella progettazione	3
2	<i>Packages</i>	6
2.1	Auth	6
2.2	Profile	7
2.3	Portfolio	8
2.4	Content	9
2.5	Institution	10
2.6	Share	11
2.7	Search	12
2.8	Domain Models	13

1 Introduzione

1.1 Compromessi e Scelte nella progettazione

Per la realizzazione del sistema si è scelto un approccio modulare basato su una netta separazione tra **client** e **server**, in maniera tale da rendere il sistema scalabile, manutenibile e semplice da implementare. Il sistema segue un'architettura **Client-Server** a più livelli, in cui l'applicazione lato client (Flutter) comunica con il back-end (Spring Boot) esclusivamente tramite chiamate REST su protocollo HTTP, mentre il back-end si occupa della logica applicativa e della persistenza dei dati su database relazionale.

I livelli logici presenti nel sistema si possono riassumere in:

- **Presentation layer (Flutter)**, per la creazione delle interfacce utente multiplatforma (Android, iOS, Web e Desktop).
- **State management (Riverpod)**, per la gestione reattiva dello stato dell'applicazione lato client e la comunicazione con i servizi API.
- **Controller layer (Spring MVC)**, per esporre gli endpoint REST e ricevere le richieste HTTP.
- **Service layer**, per implementare la logica di business del sistema.
- **Persistence layer (Spring Data JPA / Hibernate)**, per la creazione e manipolazione dei dati sul database PostgreSQL.

Per quanto concerne l'interfaccia grafica si è scelto di utilizzare il framework **Flutter**. L'utilizzo di widget componibili rappresenta un notevole vantaggio in quanto permette estrema riusabilità del codice ed in generale una maggiore leggibilità in fase di scrittura e debugging. Inoltre, con un unico codice sorgente Dart è possibile distribuire l'applicazione su più piattaforme.

Per la realizzazione del front-end sono stati utilizzati i seguenti moduli principali:

- **flutter (SDK Dart ^3.10.3)**: framework per la creazione di interfacce utente native compilate multiplatforma.
- **flutter_riverpod: ^3.3.1**: libreria per la gestione dello stato dell'applicazione basata su provider, che consente un flusso di dati reattivo e testabile.
- **dio: ^5.7.0**: client HTTP che permette di effettuare richieste come GET, POST, PUT e DELETE verso il back-end.
- **file_picker: ^11.0.2**: modulo che permette la selezione di file locali (audio, video, spartiti, curriculum) da associare ai contenuti del portfolio.

- **flutter_lints: ^6.0.0**: insieme di regole di analisi statica per garantire la qualità e la coerenza del codice.

L'applicazione front-end è organizzata secondo un'architettura a livelli: domain/models (modelli di dominio), data/services (servizi di accesso alle API), providers (gestione dello stato con Riverpod) e presentation/screens (schermate dell'interfaccia). È inoltre presente un sistema di theming centralizzato (core/theme) con supporto a tema chiaro e scuro.

Per quanto riguarda il lato server, si è scelto di usare **Java 21** con il framework **Spring Boot 4.1.0**, che permette una rapida configurazione e la gestione delle dipendenze tramite injection. Il motivo principale di questa scelta è legato alla robustezza dell'ecosistema Spring e alla sua capacità di gestire in modo strutturato sicurezza, validazione e persistenza. Il back-end è organizzato in package funzionali (auth, profile, content, portfolio, search, share), ciascuno suddiviso in Controller, Service, Repository ed Entity.

Per la realizzazione del back-end sono stati utilizzati i seguenti moduli:

- **spring-boot-starter-webmvc**: framework per l'esposizione degli endpoint REST e la gestione del routing lato server.
- **spring-boot-starter-data-jpa**: astrazione su Hibernate per la persistenza degli oggetti sul database relazionale.
- **spring-boot-starter-security**: framework per la gestione dell'autenticazione e dell'autorizzazione.
- **spring-boot-starter-validation**: libreria per la validazione dei dati in ingresso tramite annotazioni (@NotBlank, @Email, ecc.).
- **spring-boot-starter-mail e angus mail**: Invio email
- **jjwt (jjwt-api, jjwt-impl, jjwt-jackson) 0.12.6**: libreria per la generazione e la verifica dei token JWT usati nell'autenticazione stateless.
- **lombok**: libreria per la riduzione del codice boilerplate tramite annotazioni (getter, setter, builder).
- **postgresql**: driver JDBC per la connessione al database PostgreSQL in produzione.
- **h2**: database in-memory utilizzato in fase di test.
- **spring-boot-devtools**: strumenti a supporto dello sviluppo (hot reload).

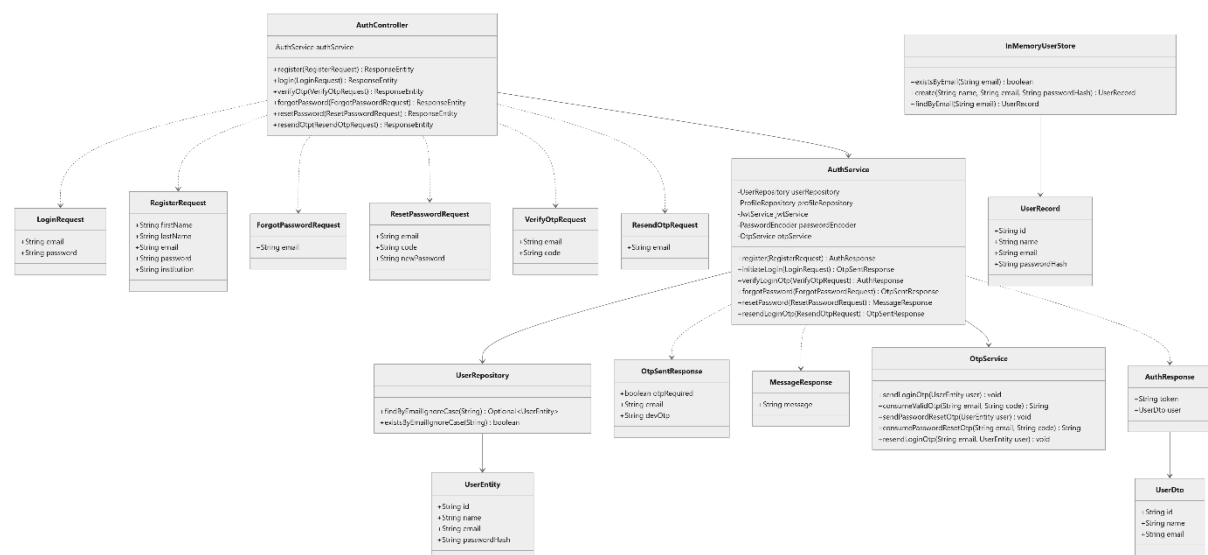
Per l'autenticazione si è scelto un approccio **stateless basato su JWT**: al login (o alla registrazione) il server genera un token firmato che il client allega nell'header Authorization: Bearer <token> di ogni richiesta successiva. Le

password sono salvate come hash tramite **BCrypt**. Le sessioni server-side non vengono mantenute (`SessionCreationPolicy.STATELESS`), riducendo il carico sul server e semplificando la scalabilità.

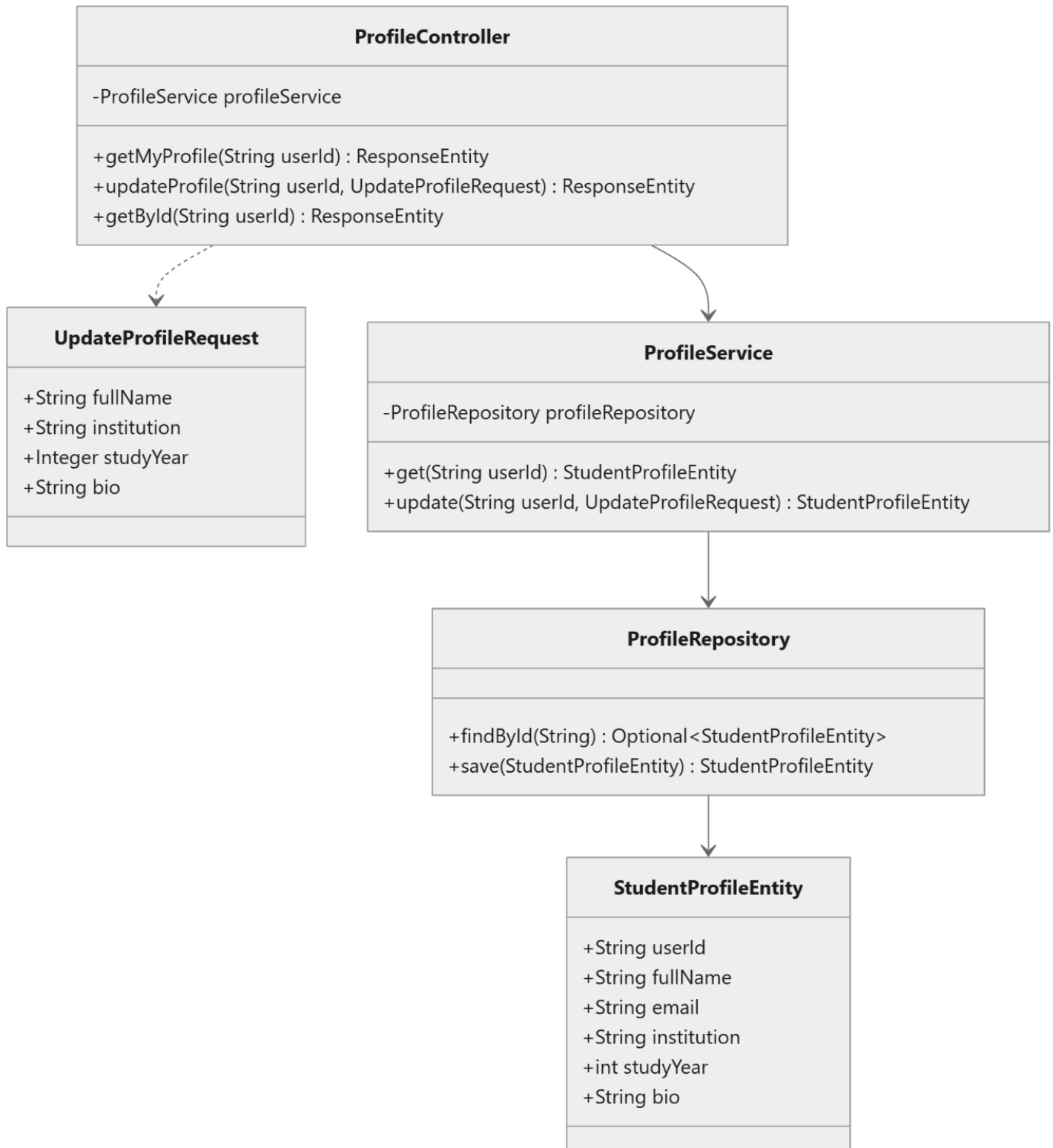
2 Packages

Per un'organizzazione ordinata e semplice durante lo sviluppo del software, si è scelto di dividere le componenti in diversi package. Il software è stato suddiviso in 2 parti: ciò che riguarda l'interfaccia con il client (front-end Flutter) e ciò che riguarda i meccanismi di esecuzione del sistema (back-end Spring Boot).

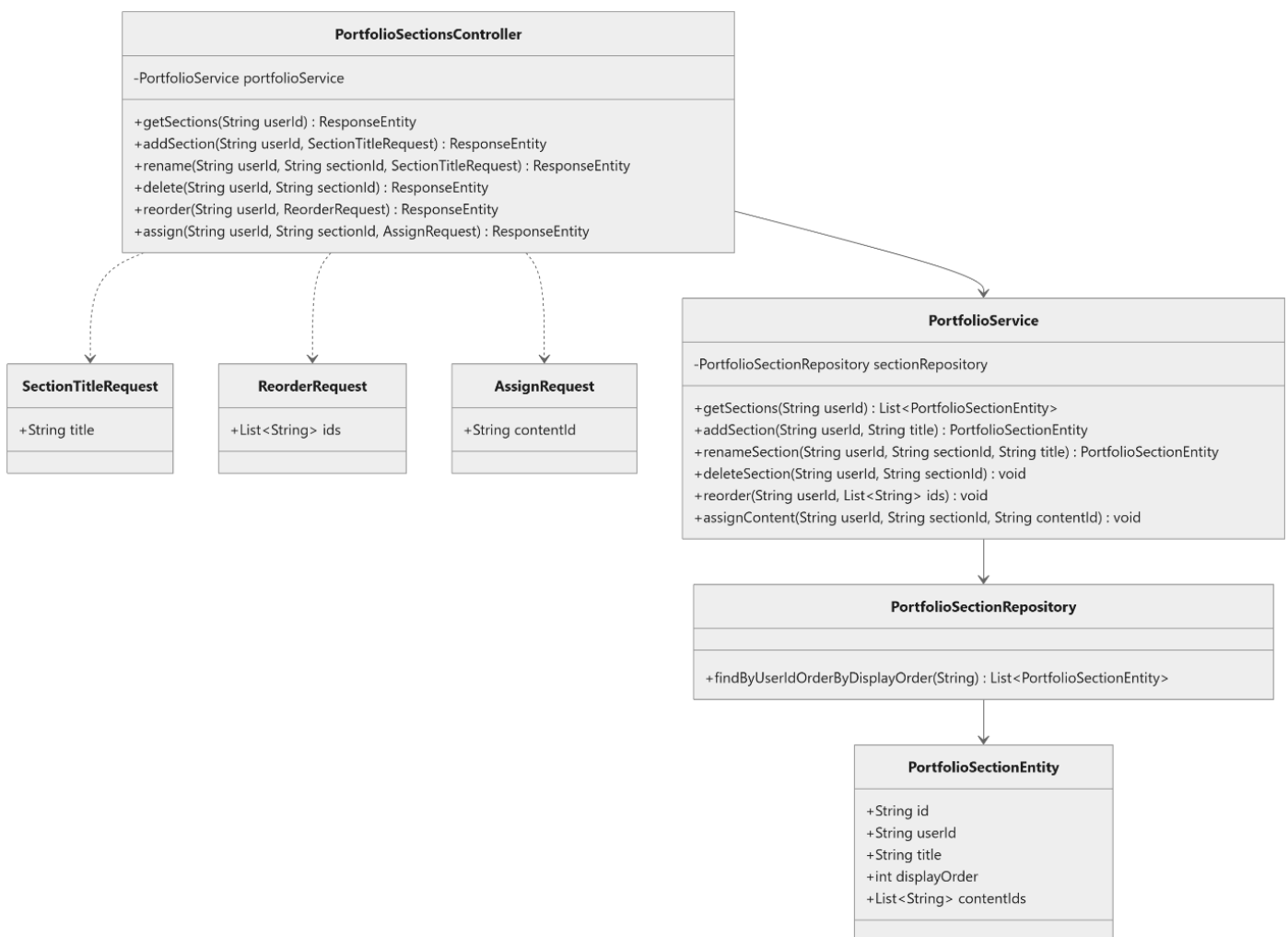
2.1 Auth



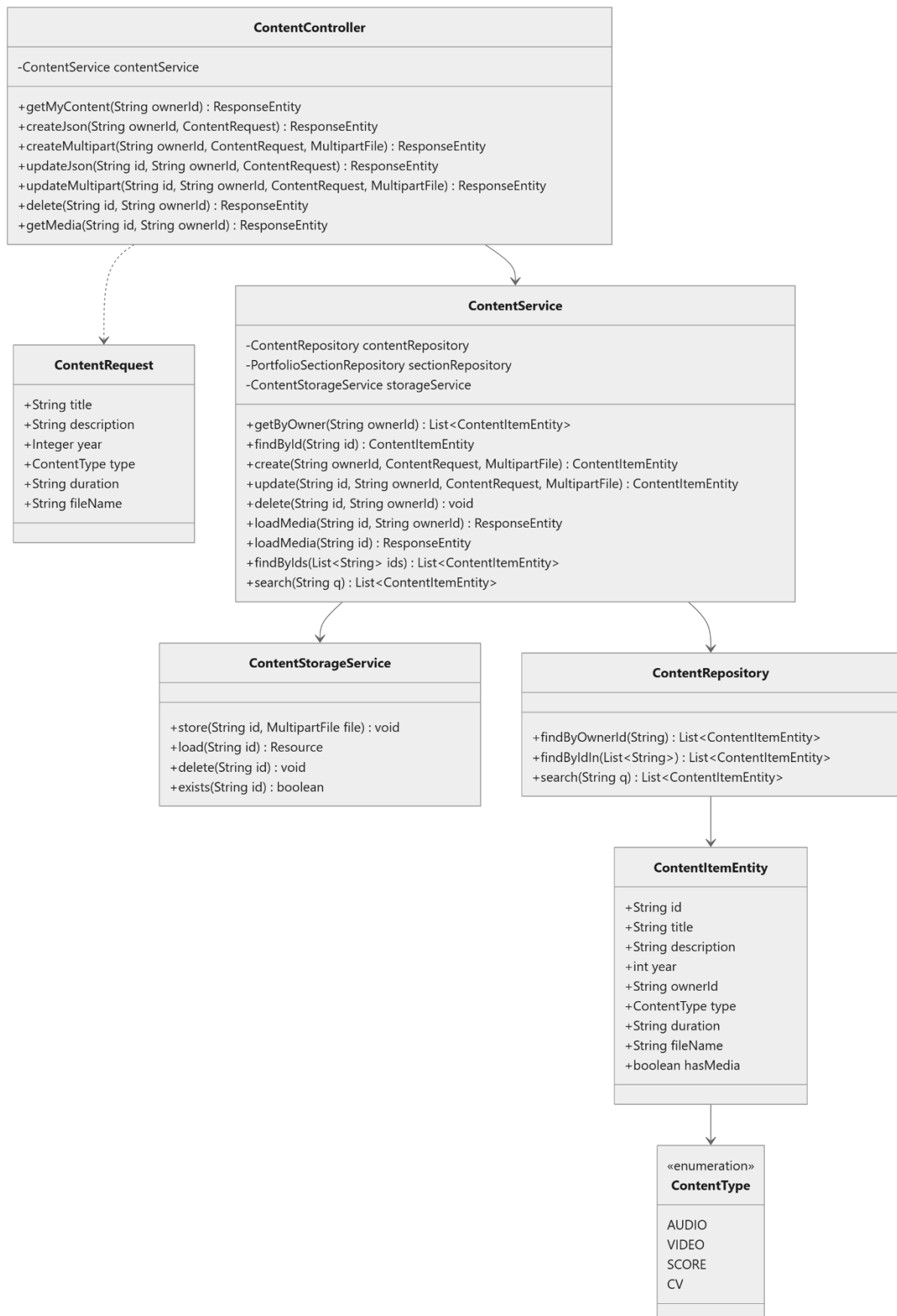
2.2 Profile



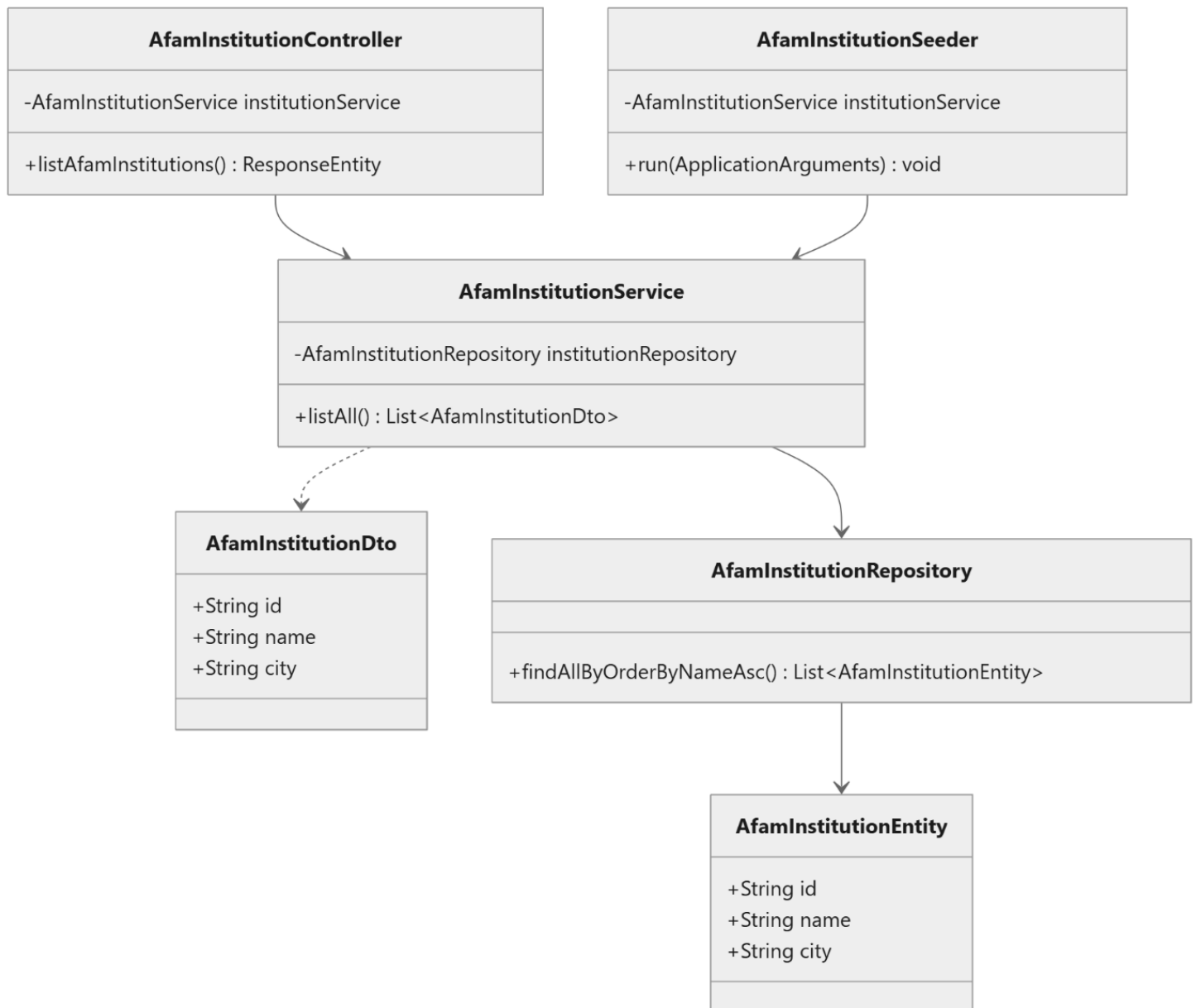
2.3 Portfolio



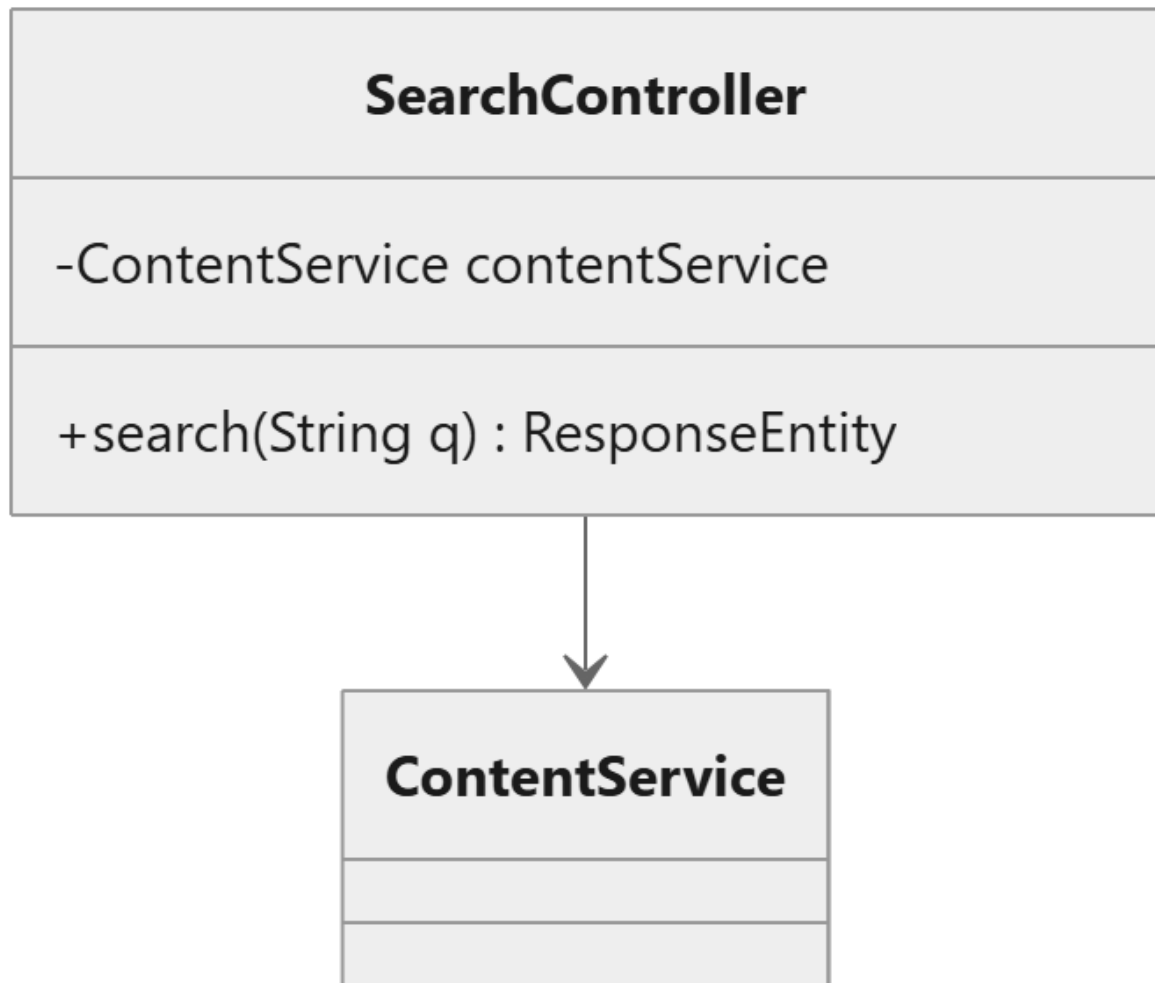
2.4 Content



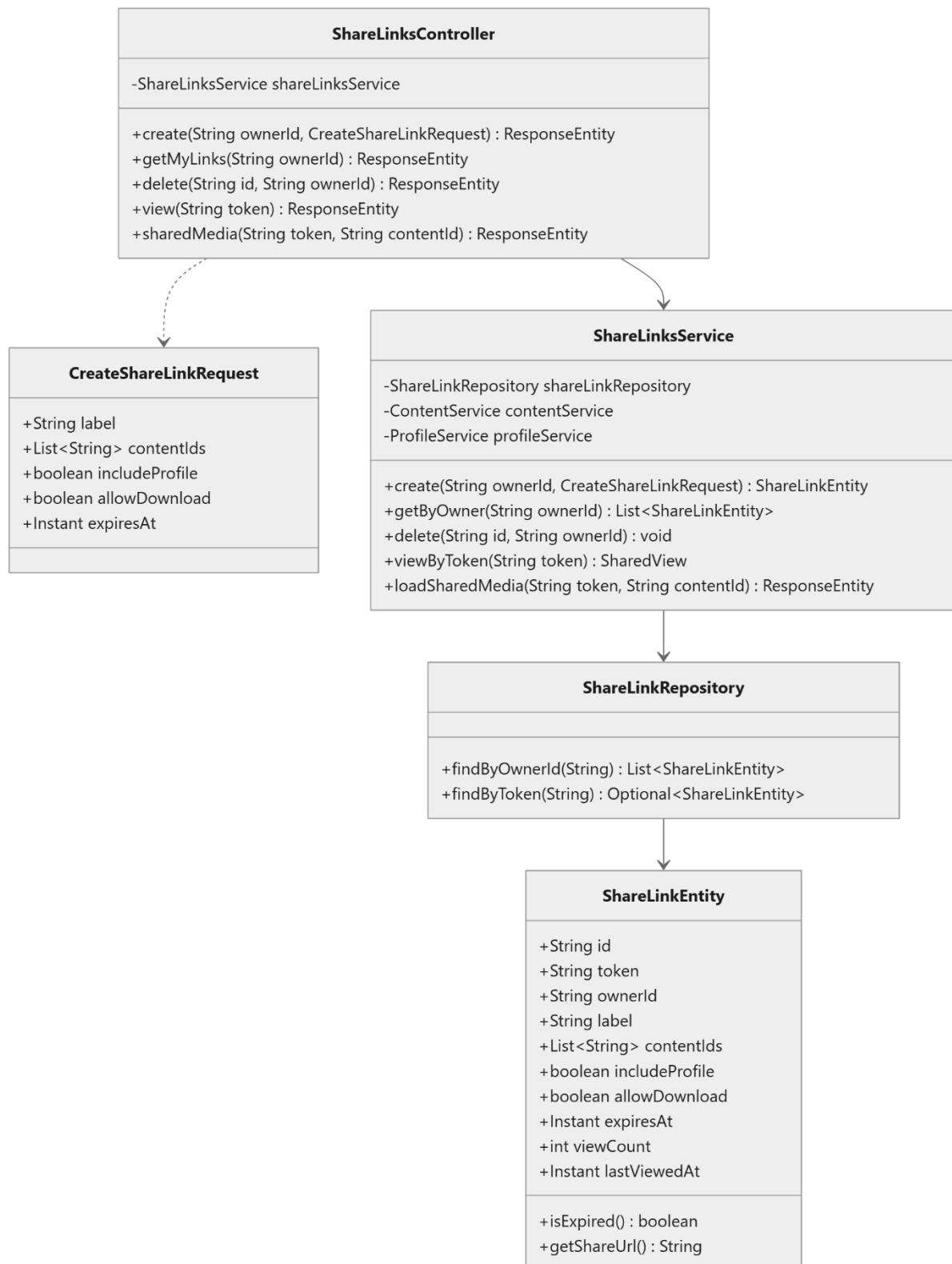
2.5 Institution



2.6 Share



2.7 Search



2.8 Domain Models

